Statičko testiranje



Autor: Goran Savić

* slajdovi su kreirani na osnovu knjige *Spillner, A., Linz, T., Schaefer, H.,* ***Software Testing Foundations****, 4th Edition, Rocky Nook, 2014*

Statičko testiranje

- Statičko testiranje podrazumeva verifikaciju test objekta analizom njegovog sadržaja
  - za razliku od dinamičkog testiranja koje verifikaciju vrši izvršavanjem test objekta korišćenjem određenih test podataka
- Može biti
  - manuelno
    - učesnici u projektu vrše analizu programa
  - automatsko
    - softverski alati vrše analizu programa
    - moguće samo za mašinski čitljive test objekte sa formalnom strukturom

Statičko testiranje

- Cilj statičkog testiranja
  - pronalaženje nedostataka u funkcionalnostima, ali i dizajnu aplikacije
  - rano otkrivanje ovih nedostataka smanjuje troškove kasnijih aktivnosti u životnom ciklusu aplikacije
- Statičkim testiranjem se može utvrditi da
  - arhitektura nije proširiva,
  - specifikacija zahteva nije ispoštovana
  - da implementacija nije usklađena sa standardima ...
- Dodatno, neki nedostaci u programu se teško otkrivaju dinamičkim testiranjem
  - npr. nedeterministička ponašanja, konkurentni problemi, *data race*

Manuelna evaluacija



Manuelna evaluacija

- Odnosi se na analiziranje dokumenata u projektu od strane učesnika
  - dokument je najčešće sam kod, ali može biti i specifikacija zahteva, model sistema, model podataka, plan aktivnosti, ...
- Različite tehnike analize
  - razlikuju se po intenzitetu, formalnosti, potrebnim resursima i ciljevima
  - ne postoji jedinstvena terminologija za ove različite tehnike analize

Pregled (*Review*)

- *Pregled* je generalni termin koji se ovde koristi za sve tehnike manuelnog statičkog testiranja
  - u praksi se najčešće koristi *review* ili *inspection*
  - u pregledu najčešće učestvuje više ljudi
    - *peer review*



Prednosti

- Moguće prednosti pregleda
  - kao i svako otkrivanje nedostataka, poboljšava dalji proces razvoja
    - što se ranije nedostaci utvrde, tim bolje
  - kvalitetnije rešenje jer je u razvoj rešenja uključeno više ljudi
  - učesnici uče jedni od drugih
  - formalizacija i dokumentovanje rešenja
    - autor mora da iznese rešenje tako da drugim učesnicima bude čitljivo i razumljivo, kao i da obrazloži odluke donete pri razvoju
  - odgovornost se raspoređuje na sve učesnike
- Iako troši resurse, pregled je neizostavan u razvoju jer značajno smanjuje broj kasnijih nedostataka i troškove otklanjanja

Problemi

- Mogući problemi u pregledu
  - uglavnom su psihološke prirode
  - autor može imati utisak da je predmet analize i kritike on i njegov rad, a ne dokument



- Problemi se mogu ublažiti
  - jasnim ciljem i dobrim upravljanjem svakim pregledom
  - dobrim izborom učesnika u pregledu

Faze pregleda

- Iako postoje različiti tipovi pregleda, obično postoje sledeće faze (implicitno ili eksplicitno izražene)
  1. planiranje
  2. informisanje
  3. samostalna priprema
  4. sastanak
  5. ispravke
  6. revizija

Planiranje pregleda

- Na generalnom nivou
  - na nivou projekta se mora doneti odluka koji dokumenti se pregledaju
  - mora se u plan projekta uvrstiti i dinamika i troškovi tih pregleda
- Na nivou jednog pregleda
  - pripremiti dokumente koji su predmet pregleda
  - izabrati učesnike u pregledu
  - obezbediti da je dokument spreman za pregled
    - da je završen na nivou predviđenom za tu fazu projekta
  - odrediti mesto i vreme sastanka i druge detalje potrebne za održavanje pregleda

Informisanje o pregledu

- Pre održavanja sastanka je potrebno
  - obavestiti učesnike o održavanju pregleda
  - obavestiti učesnike o temi i cilju pregleda
  - obezbediti učesnicima pristup dokumentima koji su tema pregleda
  - dati učesnicima dodatne informacije o dokumentima, ukoliko nisu potpuno upućeni
- Ovo je moguće realizovati pisanim pozivom ili kratkim grupnim sastankom

Samostalna priprema za pregled

- Svaki učesnik bi trebao da se pripremi za sastanak analizom dokumenata koji su tema pregleda
  - trebao bi da pripremi komentare, zamerke i pitanja za druge učesnike

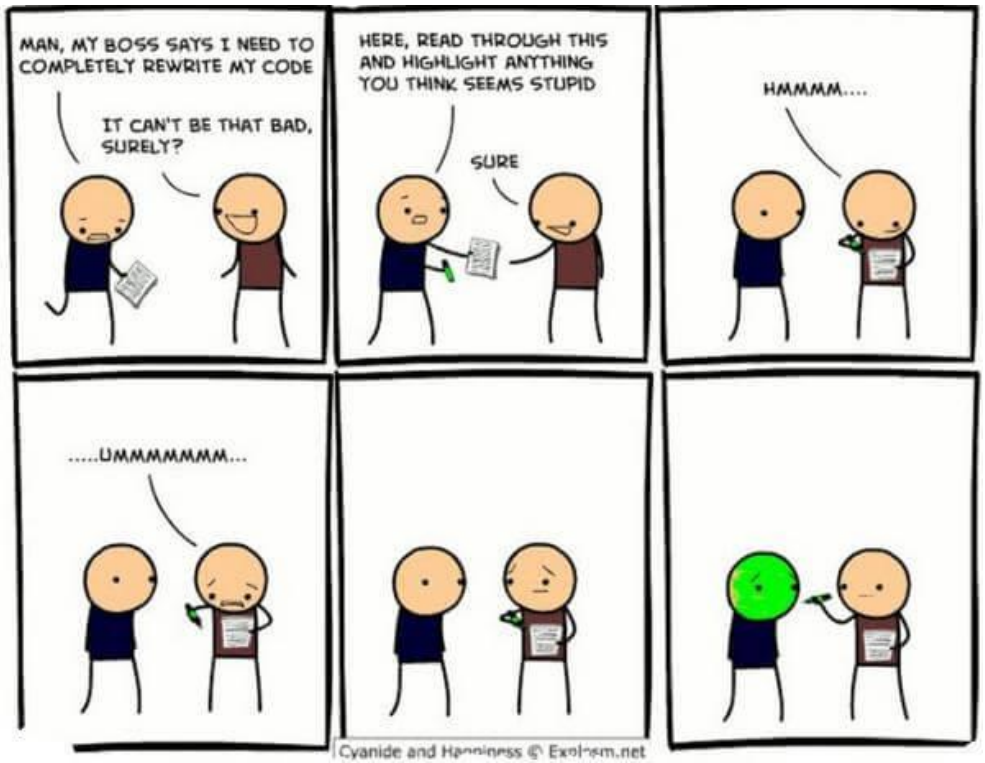


Sastanak

- Sastanak je centralna aktivnost u pregledu
- Istovremeno je i najveći izazov
  - potrebno je doneti odluke kroz diskusiju učesnika koji imaju različite stavove (nekad čak i ciljeve)
  - neophodno je sastanak unapred vremenski limitirati da bi dao rezultat
- Važno je da jedna osoba vodi sastanak
  - obezbeđuje da sastanak ne odstupi od cilja i da komunikacija bude konstruktivna i sa što manje konflikta
  - korisno je voditi beleške
- Sastanak mora završiti zaključkom
  - donosi se odluka da li se dokument prihvata, ispravlja ili odbacuje

Ispravke

- Nakon sastanka ili na samom sastanku donosi se odluka o izmenama u pregledanom dokumentu
  - najčešće je autor odgovoran da izvrši dogovorene izmene u dokumentu



Revizija

- Nakon ispravki neophodno je izvršiti reviziju nove verzije dokumenta
- Mora se obaviti novi pregled
  - procedura drugog pregleda može biti nešto kraća sa fokusom samo na modifikovane delove
- Potrebno je evaluirati sam proces pregleda kako bi naredni pregled ispravio nedostatke u proceduri
  - identifikovati faktore koji mogu naredni pregled učiniti korisnijim i efikasnijim

Uloge učesnika

- Možemo identifikovati nekoliko klasa učesnika
  - menadžer
    - upravlja procesom verifikacije sistema
    - upravlja pregledima na nivou generalnog plana i mesta pregleda u dinamici projekta
    - ne mora da učestvuje u pregledima jer često nije upućen u tehničke detalje koji se ne pregledima analiziraju
  - moderator
    - zadužen je za sprovođenje pregleda
    - vodi sve faze pregleda
    - evidentira dokumentaciju o pregledu
    - potrebno je da ima društvene veštine zbog usklađivanja učesnika u pregledu
    - trebao bi da zauzme neutralan stav na samom pregledu

Uloge učesnika

- Tipovi učesnika (nastavak sa prethodnog slajda)
  - autor
    - osoba koja je kreirala dokument koji je predmet pregleda
    - kod timskog rada, može biti delegiran jedan od autora da učestvuje u pregledu ili prisustvuju svi
    - u pravilu najteže prihvata pregled jer se njegov rad evaluira
  - recenzenti
    - osobe koje vrše pregled dokumenta
    - identifikuju nedostatke u dokumentu
    - moraju da imaju tehničko znanje
    - mogu predstavljati različite aspekte projekta (ekonomski aspekt, arhitekturu, dizajn, zahteve, ...)
  - beležnik
    - zadužen da evidentira informacije iz pregleda (probleme, stavove, odluke, zaključke)

Uloge učesnika

- U praksi često isti ljudi preuzimaju različite tipove uloga
  - npr. menadžer je istovremeno i moderator, ali i recenzent koji aktivno učestvuje u donošenju odluka u pregledu
  - takođe, za vođenje beležaka može biti zadužen neko od članova tima
    - važno je da svi imaju uvid u zaključke pregleda

Tipovi pregleda

- Razlikujemo dve grupe pregleda
  - pregledi namenjeni analizi karakteristika projekta
    - da li su funkcionalni i nefunkcionalni zahtevi realizovani
  - pregledi namenjeni analizi samog procesa razvoja projekta
    - analiziraju kvalitet upravljanja projektom, ispunjenost plana projekta i sl.

- Iako nema opšteprihvaćene klasifikacije, u nastavku je jedna moguća klasifikacija tipova pregleda namenjenih analizi karakteristika projekta

Prolazak kroz dokument (*walkthrough)*

- *Walkthrough* je neformalan pregled sa ciljem nalaženja nedostataka u dokumentu
  - vrši se kroz grupni sastanak
- Cilj je i informisanje učesnika o delovima projekta
- Vrši se bez posebne pripreme i planiranja
- Nema formalnu proceduru
  - svodi se na razmenu mišljenja sa kolegama
  - brzo i lako se organizuje

Inspekcija

- Najviše formalan tip pregleda
- Prati unapred određenu proceduru
  - uloge su striktno podeljene
  - recenzenti imaju spisak stavki koje proveravaju
  - pregled fokusiran na predefinisane aspekte
- Cilj je usko postavljen na pronalaženje nedostataka u dokumentu
- Prikupljeni podaci se formalno evidentiraju
  - mogu se koristiti za poboljšanje procesa razvoja i naredne inspekcije

Tehnički pregled

- Tehnički pregled je fokusiran na tehničke aspekte sistema
  - usklađenost sa specifikacijom
  - ispunjenost svrhe zbog koje se razvija
  - usklađenost sa standardima
- Osnov za pregled je isključivo zvanična specifikacija zahteva
- Recenzenti poseduju tehničko znanje
- Procedura može biti više ili manje formalno sprovedena
  - kod formalnog tehničkog pregleda, svi rezultati sve formalno evidentiraju

Neformalni pregled

- Svaki pregled dokumenta od strane osobe koja nije autor može se smatrati neformalnim pregledom
- Različite faze pregleda su najčešće samo implicitno izražene
- Ne mora da uključuje sastanak

Izbor tipa pregleda

- Zavisi od konkretnog slučaja
- Neke smernice za izbor
  - da li rezultati pregleda trebaju biti formalno dokumentovani
  - koliko je komplikovano organizovati grupni lični sastanak sa učesnicima
  - koliko tehničkog znanja je potrebno da učesnici poseduju
  - kakav je odnos resursa potrebnih za pripremu pregleda i koristi od rezultata pregleda
  - da li je predmet pregleda formalno specificiran da bi mogao biti analiziran programski
  - koliko je menadžment projekta obezbedio resursa za preglede

Programska statička analiza



Programska statička analiza

- Cilj je pronalaženje nedostataka, ali i generalno unapređenje kvaliteta sistema
  - isto kao i kod manuelne analize
- Ovog puta analizu vrši programski alat
- Većina alata koji se koriste pri razvoju aplikacije (editori koda, grafički alati za modelovanje, ...) vrše analizu ispravnosti dokumenta
  - korisnik dobija komentare, upozorenja, greške
- Koristi se u kombinaciji sa manuelnim pregledima
  - pre pregleda, dokument treba biti programski pregledan kako bi se smanjio broj nedostataka za ručnu analizu

Programska statička analiza

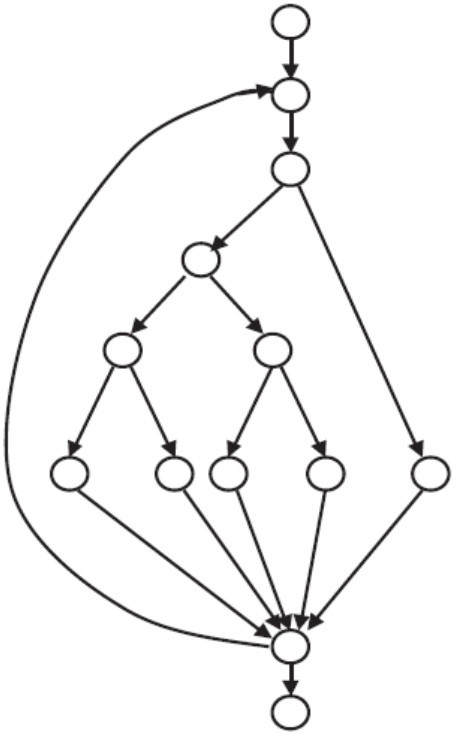
- Programska analiza ima svoja ograničenja
  - većina nedostataka se može uočiti samo dinamičkim testiranjem
    - izvršavanjem test objekta
  - dokument mora biti mašinski čitljiv i imati strukturu
  - i pored stalnog usavršavanja, neke nedostatke i dalje može da uoči samo čovek
    - ovo se posebno odnosi na to kada se sa nivoa tehničkih detalja pređe na širu sliku
    - dizajn šabloni, arhitektura, proširivost, lakoća održavanja, ...

Programska statička analiza

- Kompajler je najznačajniji alat za programsku analizu koda
  - prepoznaje sintaksne greške, ali i druge nedostatke (nekorišćene promenljive, nedostupan kod, ...)
- Pored kompajlera, koriste se i posebni alati, tzv. analizatori
- Programskom statičkom analizom se mogu utvrditi
  - sintaksni nedostaci
  - odstupanja od konvencija, pravila i standarda
    - pravila su često ugrađena u IDE za razvoj programa
  - nedostaci u kontroli toka programa
  - nedostaci u toku podataka u programu

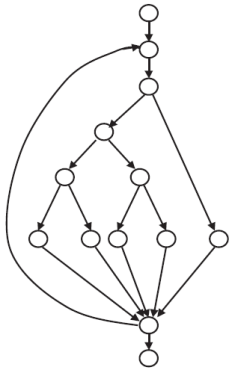
Analiza toka programa

- Programski utvrđuje nepravilnosti u toku programa
- Izgrađuje graf toka programa
- Može da
  - utvrdi da program ni u jednom scenariju neće izvršiti određeni deo koda (*unreachable code*)
  - utvrdi da tok programa nije u skladu sa konvencijama
    - npr. više tačaka izlaza iz funkcije
  - identifikuje potencijalne neželjene greške
    - npr. nedostatak *break* izraza u *case* labeli *switch* izraza



Merenje kompleksnosti koda

- Programski se može proceniti i kompleksnost koda
  - prebrojavanjem broja linija koda
  - izračunavanjem ciklomatskog broja
- Ciklomatski broj
  - daje informaciju o kompleksnosti strukture koda
    - na osnovu broja mogućih grana u izvršavanju koda
  - računa se na osnovu grafa toka programa formulom
    - $v(G) = e - n + 2$
    - e – broj grana u grafu
    - n – broj čvorova u grafu



SonarQube



- Alat za programsku analizu kvaliteta koda koji prepoznaje
  - odstupanje od standarda i konvencija
  - potencijalne nedostatke
  - kompleksnost koda
  - dupliranje koda

- Podržava različite programske jezike, uključujući i Java i JavaScript

SonarQube



- Instalacija
  - potrebno je preuzeti i instalirati SonarQube server
  - može i na lokalnom računaru
- Pokretanje na Windows OS
  - Pokrenuti Command Prompt kao administrator
  - Pokrenuti SonarQube server
    - `<sonar_dir>/bin/windows-x86-64/StartSonar.bat`
- Server je dostupan na
  - http://localhost:9000
  - ako je instaliran na lokalnom računaru
  - Ulogujemo se sa korisničkim imenom admin i šifrom admin
  - Na stranici User -> My Account -> Security izgenerisati authentikacioni token

SonarQube



- Analiza programskog koda
  - u folderu koji sadrži kod koji je potrebno analizirati se kreira fajl `sonar-project.properties`
  - primer fajla
    - `students-testing/sonar-project.properties`
  - iz komandne linije se pozicioniramo u folder sa izvornim kodom
  - Izvršimo
    - `mvn clean verify sonar:sonar -Dsonar.projectKey=Test-students -Dsonar.projectName='TestStudents' -Dsonar.host.url=http://localhost:9000 -Dsonar.token=abcdefghijkl`
    - rezultati analize su dostupni u *browseru* na URL-u servera
    - npr. localhost:9000

SonarLint

- Eclipse i IntelliJ Plugin za SonarQube
- Omogućuje prikaz osnovnih rezultata Sonar analize u IDE
- Potrebno je imati instaliran SonarQube server i povezati se na njega u konfiguraciji SonarLint u IDE